Macroeconomic Forecasting with VAR, VECM, and Threshold Models

Cuauhtemoc Daniel Puente Cavazos

In [37]:
```
#### GOOGLE COLAB ######
%pip install rpy2 --quiet

####### ANACONDA #######
#import sys
#!conda install -c conda-forge --yes --prefix {sys.prefix} r
#!conda install -c conda-forge --yes --prefix {sys.prefix} rpy2
#import os
#os.environ["R_HOME"] = f"{os.environ['CONDA_PREFIX']}\\Lib\\R"
```

```
Note: you may need to restart the kernel to use updated packages.
```

Load R packages

In [5]:
```
# Suppress superfluous warnings, only display errors
from rpy2.rinterface_lib.callbacks import logger as rpy2_logger
import logging
rpy2_logger.setLevel(logging.ERROR)

# Import rpy2 and activate automatic R conversion of Python objects
import rpy2
import rpy2.robjects as robjects
from rpy2.robjects.packages import importr, data
from rpy2.robjects import pandas2ri
pandas2ri.activate()
from rpy2.robjects import numpy2ri
numpy2ri.activate()

# Import R packages
base = importr('base')
```

```
utils = importr('utils')
utils.chooseCRANmirror(ind=1)
utils.install_packages('urca')
urca = importr('urca')
utils.install_packages('tsDyn')
tsDyn = importr('tsDyn')
```

```
The downloaded binary packages are in
	/var/folders/w6/k251t5zj7v30rv6xqgb7j4s80000gn/T//Rtmpu55Cc7/downloaded_packages

The downloaded binary packages are in
	/var/folders/w6/k251t5zj7v30rv6xqgb7j4s80000gn/T//Rtmpu55Cc7/downloaded_packages
```

Load libraries

In [6]:
```
import numpy as np, pandas as pd, pandas_datareader.data as web, datetime as dt
import matplotlib.pyplot as plt, matplotlib.dates as mdates, statsmodels.api as sm
import statsmodels.tsa as st
from tabulate import tabulate
import warnings
warnings.filterwarnings("ignore")
```

1. Data

In [7]:
```
Macro_data_can = pd.read_csv('Macro_data_can.csv')
Macro_data_can['Index'] = pd.to_datetime(Macro_data_can['Index'], format="%Y-%m-%d")
Macro_data_can.set_index('Index', inplace=True)

Macro_data = Macro_data_can
Macro_data.dropna(inplace=True) # balance the panel by removing rows with missing values
Macro_data = Macro_data.loc["2015-01-01":] # starting date for the data
vnames = Macro_data.columns
dat = Macro_data.diff().dropna()
```

Plot the data

2. Model Selection

2.1 Johansen test

In [9]:
```
jotest = urca.ca_jo(Macro_data, type="trace", ecdet="trend", spec="transitory")
jta = robjects.r.attributes(jotest) # extract attributes from an RS4 object jotest
stat = jta.rx2('teststat')
cval = jta.rx2('cval')
```

In [11]:
```
k = Macro_data.shape[1]-1
rnames = np.array([["r = {}".format(i) if i==0 else "r <= {}".format(i)] for i in range(k,-1,-1)])
jres = np.concatenate((rnames, stat.reshape(-1,1), cval), axis=1)
table = jres.tolist()
print(jta.rx2('test.name'))
print(tabulate(table, headers=["test stat", "10pct", "5pct", "1pct"], floatfmt=".2f"))
```

```
['Johansen-Procedure']
          test stat    10pct    5pct    1pct
------  -----------  -------  ------  ------
r <= 3         5.44    10.49   12.25   16.26
r <= 2        19.03    22.76   25.32   30.45
r <= 1        48.50    39.06   42.44   48.45
r = 0        112.99    59.14   62.99   70.05
```

We will reject 0 and 1, but do not reject 2. So R = 2

2.2 VAR, TVAR, VEC time-series validation MSE

In [24]:
```
robjects.globalenv["dat"] = dat

for lag in range(1, 2+1):  # range is [1, 2]
    print("lag:")
    print(lag)
    robjects.globalenv["lag"] = lag

    for m in range(1, 3+1): # range is [1, 2, 3]
        TT = dat.shape[0]
        T1 = int(np.floor(0.5*TT))  # start at 50% of the sample size
        step = 12  # forecast data horizon for MSE
```

```
        robjects.globalenv["step"] = step
        tseq = np.arange(start=T1, stop=TT+1, step=step)
        tseq = tseq[:-1]
        MSE_t = np.zeros((tseq[-1] + step - T1, len(vnames)))  # initialize
        MSE_t = pd.DataFrame(MSE_t, columns=vnames)

        for j in tseq:
            robjects.globalenv["j"] = j

            # VAR model
            if m == 1:
                model = sm.tsa.VAR(dat.iloc[:j-1])
                model_fit = model.fit(maxlags=lag)
                fcst = model_fit.forecast(y=np.array(dat[-lag:]), steps=step)
                r_code = '''stats::predict(tsDyn::lineVar(data=dat[1:j-1,], lag=lag,
                           model="VAR", I="diff"), n.ahead=step)'''
                fcst = robjects.r(r_code)

            # TVAR model
            elif m == 2:
                r_code = '''stats::predict(tsDyn::TVAR(data=dat[1:j-1,], lag=lag,
                           model="TAR", nthresh=1, trace=F), n.ahead=step)'''
                fcst = robjects.r(r_code)

            # VEC model
            elif m == 3:
                model = sm.tsa.VECM(dat.iloc[:j-1], dates=dat.index[:j-1], k_ar_diff=lag, coint_rank=2)
                model_fit = model.fit()
                fcst = model_fit.predict(steps=step)
                r_code = '''stats::predict(tsDyn::lineVar(data=dat[1:j-1,], lag=lag,
                           r=2, model="VEC"), n.ahead=step)'''
                fcst = robjects.r(r_code)

            js = j + step - 1
            MSE_t.iloc[j-T1:js-T1+1] = (dat.iloc[j-1:js].to_numpy() - fcst)**2

        if m == 1:
            print("VAR")
        elif m == 2:
            print("TVAR")
        elif m == 3:
            print("VEC")
```

```
        MSE = MSE_t.mean(axis=0).to_numpy().reshape(1, -1)
        MSE = pd.DataFrame(MSE, columns=vnames)
        MSE = MSE.round(decimals=4)
        MSE_str = MSE.to_string(index=False) # for neater printing
        print(MSE_str)
        print(" ")
```

```
lag:
1
VAR
   CPI       GDP  Unemployment  Target.Rate
0.2047 1594.3659        0.7894       0.1616

TVAR
   CPI       GDP  Unemployment  Target.Rate
0.2683 1603.1786        0.7977       0.0917

VEC
   CPI       GDP  Unemployment  Target.Rate
0.2034 1641.0951        0.8097       0.1563

lag:
2
VAR
   CPI       GDP  Unemployment  Target.Rate
0.1923 1636.0569        0.7945       0.1269

TVAR
   CPI       GDP  Unemployment  Target.Rate
0.2889 1451.0615        0.8228       0.1026

VEC
   CPI       GDP  Unemployment  Target.Rate
0.1979 1629.3769        0.7857       0.1246
```

The model that yields the smallest time series validation **MSE** for each variable is ***VAR*** (lag 2) for *CPI* (0.1923), ***VEC*** (lag 2) *Unemployment* (0.7857), ***TVAR*** (lag 2) for *GDP* (1451.0615), and ***TVAR*** (lag 1) for the *Target Rate* (0.0917), These ones give the lowest **MSE** values compared to other ones.

.